Software Engineering & Design Principles Lab

Experiment 2 — Feasibility Report

Identification of Problem, Objectives, Scope and Feasibility Analysis

Student / Team Details

Particular	Details
Experiment No.	2
Project Title	AI-Based Attendance Monitoring System
Student Name(s)	Arjun Agrawal, Pankaj Bhargav, Pankaj Chaudhary, Prachi
Roll No(s).	2400290310042, 2500290109018, 2500290109019, 2500290109020
Semester & Section	5th Sem , Section-D
Date of Submission	21st August, 2026

1. Problem Identification

1.1 Background / Context

This project sits in the Education domain, specifically classroom/institutional administration. Every college takes attendance every single day, in every class, for every student — it's one of the most repeated manual tasks on campus, yet most institutes (including ours) still do it the same way it was done twenty years ago: someone calls out names or passes around a sheet. That repetition is exactly what makes it a good candidate for automation using AI and computer vision.

1.2 Existing Problem

In almost every lecture, the first 5–10 minutes go into roll call or signing an attendance sheet, time that could just go into teaching. It's also easy to game: a friend can answer “present” for someone who isn't there, or sign the sheet on their behalf. Once the sheet is filled, that data usually sits on paper or in a teacher's personal spreadsheet until someone in the exam cell has to manually total it up before internal exams — by which point a student who's been skipping class for weeks is flagged far too late to actually fix anything.

1.3 Existing Method / Current System

Right now attendance is taken by verbal roll call, a paper register that gets signed and later transferred to Excel, or, in a few better-equipped institutes, RFID cards / fingerprint scanners at the entrance. All three roughly end the same way: numbers eventually land in a spreadsheet or the college ERP, but only after a manual data-entry step somewhere in the chain.

1.4 Limitations of the Existing System

1. Consumes 5–10 minutes of every class period that could be used for teaching.
2. Susceptible to proxy attendance / “buddy punching” in roll-call and card/biometric-based systems.
3. Manual paper records are error-prone, hard to search, and easily lost or damaged.

4. No real-time visibility of attendance trends for faculty, admin, or parents.
5. Card and fingerprint hardware require physical contact, cause queues, and need regular maintenance.

2. Problem Statement

Faculty and academic administrators need a faster, harder-to-cheat way to record student attendance and actually make sense of it afterward. Roll call, paper registers, and even card/biometric scanners are slow, still leave room for proxy marking, and the data they produce is scattered and painful to consolidate into a usable report. What's needed is a system that recognises students automatically from a camera feed during class, refuses to be tricked by a friend standing in, and turns that into a report the moment it's needed — not weeks later.

3. Objectives of the Project

S. No.	Objective
1	To develop an AI-based facial-recognition system that automatically marks student attendance from live classroom video/images.
2	To eliminate proxy attendance through real-time face detection, recognition, and liveness checks.
3	To reduce the time and manual effort spent on daily attendance-taking.
4	To generate real-time, exportable attendance reports and analytics for faculty and administration.
5	To integrate the attendance module with the institute's existing ERP / Learning Management System.
6	To provide a web/mobile dashboard where students, faculty, and admin can view attendance records.
7	To send automated low-attendance alerts to students, parents, and mentors.
8	To ensure the privacy and security of student facial-biometric data in compliance with data-protection norms.

4. Scope of the Project

4.1 Target Users / Stakeholders

S. No.	User / Stakeholder	Role
1	Students	Get face enrolled once; attendance is captured automatically each class; can view their own attendance record and streak.
2	Faculty / Teachers	Attendance capture starts automatically per the timetable; review and clear the low-confidence match queue; view class-wise reports.
3	Academic Admin Office	Manage master data (students, courses, timetable), monitor institute-wide attendance, generate compliance reports.

S. No.	User / Stakeholder	Role
4	IT / System Administrator	Install and maintain cameras/edge devices, manage system uptime, security, and data backups.
5	Parents / Guardians	Receive attendance and low-attendance alerts for their ward.

4.2 Functional Scope

1. Face detection and recognition-based attendance marking via a single wide-angle classroom camera, calibrated once to the room's seating layout instead of mounting a camera over every row.
2. A short liveness check (blink / slight head-turn) before marking someone present, to stop the obvious trick of holding up a friend's photo to the camera.
3. Timetable-based automatic start/stop of attendance capture, so faculty don't have to remember to open the app and tap “start session” every period.
4. Low-confidence matches (poor lighting, mask, new haircut) are held in a short review queue for the teacher to confirm, instead of being auto-marked and risking a wrong entry.
5. Automatic generation of daily, weekly, and monthly attendance reports.
6. Admin dashboard to manage student, faculty, course, and timetable data.
7. Automated low-attendance notifications to parents/mentors, paired with a simple attendance-streak view on the student's own dashboard so it isn't purely punitive.
8. Integration/API with the existing college ERP or Learning Management System.
9. Support for multiple classrooms and multiple cameras running simultaneously.

4.3 Data Scope

S. No.	Data / Information	Description
1	Student master data	Name, roll number, course/section, and a numeric facial-embedding vector generated at enrolment — the raw photo itself is not retained, only the vector, so nothing usable is exposed even if the database is ever compromised.
2	Attendance records	Date, time-stamp, subject/period, camera/location, present/absent status, and match-confidence score.
3	Faculty & timetable data	Faculty details, subjects handled, class schedule used to auto-trigger attendance sessions.
4	Reports & analytics data	Aggregated attendance percentages, trends, low-attendance flags, and student streak counters.

4.4 Out of Scope

1. Payroll or salary processing for faculty and staff.
2. A full replacement of the institute's existing ERP (the system integrates with it, not replaces it).
3. Manufacturing or supply of camera/edge-device hardware.
4. Emotion or behavioural analysis of students beyond identity recognition for attendance.

Note: The features listed above may be considered as future enhancements outside the current project scope.

5. Technical Feasibility Analysis

Technical feasibility determines whether the proposed system can be technically developed and implemented with the resources available to the team.

5.1 Proposed Technology Stack

Component	Proposed Technology
Frontend	React.js (admin/faculty web dashboard); Flutter (optional student mobile app)
Backend	Python — Django REST Framework / FastAPI
Database	MySQL / PostgreSQL for structured data; Firebase for notifications
Programming Language(s)	Python, JavaScript
Development Tools / IDE	VS Code, Jupyter Notebook, Git & GitHub, Postman
Other Technologies	OpenCV for capture, a CNN-based model (FaceNet/MTCNN or the Dlib face_recognition library) to generate face embeddings, plus a lightweight liveness-detection step; cloud storage (AWS S3 / Google Cloud) only for embeddings and metadata, not raw images

5.2 Detection Accuracy & Handling Changing Faces

Two things break a naive face-recognition setup in practice: missing faces in a crowded frame, and faces changing over time (haircut, beard, glasses, weight, or just ageing across semesters). The plan to handle both:

- Use a detector built for crowded scenes (RetinaFace / MTCNN / YOLO-face) rather than a basic Haar cascade, since classrooms have dozens of faces at different angles and distances at once.
- Sample multiple frames over the first several minutes of class instead of a single snapshot, so a student who briefly looks down or is blocked by a classmate still gets caught in a later frame.
- Use a high-enough resolution camera (1080p or better) so back-row faces aren't too small in pixels for the detector to work with.
- Enrol each student with 3–5 reference photos (different angles, with/without glasses) instead of one ID-style photo, giving a more robust starting embedding.
- When a teacher manually confirms a low-confidence match from the review queue, quietly use that confirmation to refresh the student's stored embedding, so the system adapts as their appearance changes instead of staying frozen from enrolment day.
- If a student's match confidence keeps trending down over several sessions, prompt them to re-enrol rather than letting accuracy silently degrade.

None of this claims 100% detection — a bag, a raised hand, or someone leaning out of frame will still be missed sometimes. That's exactly why the low-confidence review queue exists: anyone not detected shows up as absent-by-default, and the teacher clears it in a few taps rather than the system silently getting it wrong.

5.3 Hardware Requirements

Hardware	Requirement
Processor	Intel i5 (deployment) / i7 or better with NVIDIA GPU (model training)

Hardware	Requirement
RAM	8 GB minimum for deployment; 16 GB recommended for model training
Storage	256 GB SSD or higher for application, models, and image data
Other Hardware	One HD wide-angle camera per classroom, calibrated once to the room's seating layout (avoids the cost of a camera over every row); optional edge device (Raspberry Pi / NVIDIA Jetson Nano) for on-site inference

5.4 Software Requirements

Software	Version / Requirement
Operating System	Windows 10/11 or Linux (Ubuntu 20.04+)
IDE / Editor	VS Code / PyCharm
Database	MySQL 8.x / PostgreSQL 14+
Framework / Libraries	OpenCV, TensorFlow or PyTorch, Django/FastAPI, React.js
Other Software	Git, Docker (containerized deployment), Postman (API testing)

5.5 Technical Skills Required

- Python programming and REST API development
- Machine learning / deep learning fundamentals, particularly CNNs for computer vision
- OpenCV and face-recognition library usage
- Frontend web development (React.js) and basic mobile development
- Database design and management (SQL)
- Basic cloud deployment and Git-based version control

Technical Feasibility Assessment

Python, OpenCV, and the open-source face-recognition libraries we're planning to use are free, well documented, and mostly stuff the team has already touched in coursework or personal projects. Hardware-wise, a mid-range laptop plus an optional GPU for training and a normal HD webcam covers the pilot — nothing exotic. Free-tier cloud credits and public face datasets should be enough to get a working prototype trained and tested within a semester. The detection-accuracy and changing-face handling described above (multi-frame sampling, multi-photo enrolment, confidence-based re-training) are all achievable with open-source libraries and don't need any specialised hardware beyond what's already listed.

Conclusion: ☑ Technically Feasible ☐ Technically Not Feasible

Justification: the tools and skills needed are within what the team can reasonably pick up, and the hardware cost for a single pilot classroom is low.

6. Operational Feasibility Analysis

Operational feasibility looks at whether the system will actually get used once it's built, not just whether it can be built.

6.1 User Acceptance

Faculty should take to this fairly quickly — it removes a task nobody enjoys doing every period. Students will probably be fine with it too, as long as enrolment doesn't take forever and someone actually explains what's being stored and why before it's switched on. A short awareness session before rollout would help a lot here.

6.2 Ease of Use

For the end user this should feel almost invisible: the camera captures attendance while class happens, and faculty only need to open the dashboard afterward to glance at or correct the odd flagged entry. No technical know-how needed on their end.

6.3 Existing Workflow

It slots into the class period the same way roll call used to — it just happens automatically now. What changes is downstream: attendance percentage and defaulter lists that used to get compiled by hand at the end of the semester are now available instantly.

6.4 Training Requirements

Barely any. Faculty need maybe a 10–15 minute walkthrough on reviewing flagged entries, and IT needs a one-time session for camera/server setup.

6.5 Expected Operational Benefits

1. Significant reduction in time lost to manual attendance-taking every class.
2. Elimination of proxy attendance and associated disciplinary issues.
3. Faster, more accurate compliance reporting for university/regulatory audits.
4. Early identification of students with low attendance, enabling timely mentor/parent intervention.

Operational Feasibility Assessment

Conclusion: ☑ Operationally Feasible ☐ Operationally Not Feasible

Justification: it automates something everyone already understands, needs almost no training, and the time saved is obvious from day one.

7. Budget Estimate

This covers just the pilot — one classroom, built in-house by the team, so there's no development cost to add:

Budget Head	Item	Estimated Cost (₹)
Hardware	HD camera(s) for pilot classroom	5,000 – 8,000
Hardware	Edge device / mini-PC (optional, for on-site inference)	6,000 – 10,000

Budget Head	Item	Estimated Cost (₹)
Computing	Cloud GPU credits for model training (or free-tier academic credits)	0 – 5,000
Software	Licenses (mostly open-source: OpenCV, TensorFlow, Django, React)	0 – 2,000
Storage/Hosting	Cloud storage & hosting for pilot phase (free-tier eligible)	0 – 3,000
Development	Manpower (in-house student team)	Nil (academic project)
Contingency	Miscellaneous / buffer (≈10%)	≈ 2,000
Total (Pilot Phase)		≈ ₹13,000 – ₹30,000

Scaling this beyond one classroom mostly just means buying more cameras/edge devices and a bit more storage — the software itself doesn't need to be rebuilt.

8. Overall Feasibility Summary

Feasibility Area	Status	Justification
Technical Feasibility	Feasible	Required tech stack, hardware, and skills are accessible and within team capability.
Operational Feasibility	Feasible	Automates an existing workflow with minimal training and clear time-saving benefits.
Budget / Economic Feasibility	Feasible	Pilot deployment achievable within a modest academic budget using largely open-source tools.

Overall Conclusion: putting the technical, operational, and budget pieces together, the AI-Based Attendance Monitoring System looks feasible to build as a pilot — the tools are accessible, the hardware footprint is small, faculty won't need much training, and the benefit (no more wasted class time and no more proxy attendance) is easy to see.

9. Expected Outcome

By the end of this project, we expect a working system that marks attendance from a live camera feed in real time, refuses to be fooled by a photo or a friend answering on someone's behalf, and gives faculty and admin a report they can pull up instantly instead of compiling by hand.

10. Future Enhancements (Optional)

1. A dedicated mobile app for parents with real-time attendance and alert notifications.
2. Integration with the institute's fee-management and examination-eligibility modules.
3. Classroom engagement analytics (e.g., attention tracking) as an additional, opt-in feature.
4. Multi-campus deployment with centralized analytics for institutes with multiple branches.